# P P SAVANI UNIVERSITY

## SCHOOL OF ENGINEERING

### B.TECH FINAL PROJECT REPORT

(Hard-Bound Project Report Format)

# PARKSMART: REAL-TIME SMART PARKING RESERVATION & AUTOMATED SECURITY GATE ACCESS MANAGEMENT SYSTEM

| Particular | Details |
|---|---|
| **Project Title** | ParkSmart: Real-Time Smart Parking Reservation & Automated Security Gate Access Management System |
| **Student Name** | **KANANI MYUSI UDAYBHAI** |
| **Enrollment Number** | **23SE02CE064** |
| **Program / Branch** | B.Tech - Computer Engineering |
| **Semester** | 8th Semester |
| **Company / Organization** | P P Savani University (University UDP Project) |
| **Department / Domain** | Full-Stack Web Development & Smart Transportation Systems |
| **Mentor (Company)** | Industry Mentor / Guide |
| **Mentor (Institute)** | Internal Guide / Faculty Mentor Name |
| **Academic Year** | 2026–2027 |

P P SAVANI UNIVERSITY | SCHOOL OF ENGINEERING

# CERTIFICATE OF COMPLETION

This is to certify that **KANANI MYUSI UDAYBHAI**, Enrollment No.: **23SE02CE064**, student of **B.Tech Computer Engineering, 8th Semester**, has successfully completed the Final Year Project titled:

## "PARKSMART: REAL-TIME SMART PARKING RESERVATION & AUTOMATED SECURITY GATE ACCESS MANAGEMENT SYSTEM"

during the academic term at **P P Savani University** under the guidance and supervision of Institute Mentors.

**Date of Issue:** 12 / 10 / 2026

_____
**Mentor (Company / Industry)**

_____
**Mentor (Institute / Internal)**

# DECLARATION

I hereby declare that this project report entitled **"ParkSmart: Real-Time Smart Parking Reservation & Automated Security Gate Access Management System"** is an authentic record of our engineering work carried out during the academic year 2026–2027.

The work embodied in this project has not been submitted elsewhere for the award of any other degree or diploma. All external references, libraries, and frameworks have been duly acknowledged and cited according to academic standards.

**KANANI MYUSI UDAYBHAI**
Enrollment No: 23SE02CE064
B.Tech Computer Engineering

# LIST OF FIGURES


| Sr. No. | Figure No. | Figure Title | Type |
|---|---|---|---|
| 1 | Figure 5.1 | Overall Project Workflow | Vector Flowchart |
| 2 | Figure 6.1 | Deployed System Architecture of ParkSmart | Cloud Architecture |
| 3 | Figure 6.2.1 | Customer / Driver Dashboard Process Flow | Process Flowchart |
| 4 | Figure 6.2.2 | Admin Management Process Flow | Admin Workflow |
| 5 | Figure 6.2.3 | Security Staff Smart Auto Gate Process Flow | Decision Flowchart |
| 6 | Figure 6.3.1 | Block Diagram – Admin Operations Hub | Block Diagram |
| 7 | Figure 6.3.2 | Security Gate Scanner Engine Flowchart | Scanner Engine |
| 8 | Figure 6.3.3 | UML Class Diagram of ParkSmart Entities | UML Class Model |
| 9 | Figure 6.3.4 | Block Diagram of Customer Dashboard | Block Diagram |
| 10 | Figure 6.3.6 | Payment Gateway Sequence Diagram | Sequence Diagram |
| 11 | Figure 6.3.7 | Slot State Transition Diagram | State Machine |
| 12 | Figure 6.4 | Major Interface / UI Wireframe Design | UI Wireframe |
| 13 | Figure 8.1 | Module 1 User Authentication Interface | Implementation |
| 14 | Figure 8.2.1 | Module 2 Interactive 2D Visual Slot Grid Layout | Implementation |
| 15 | Figure 8.2.2 | Module 2 Duration & Vehicle Booking Modal | Implementation |
| 16 | Figure 8.2.3 | Module 2 Dynamic QR Digital Pass with Timer | Implementation |
| 17 | Figure 8.3 | Module 3 Security Gate Access Desk & Viewfinder | Implementation |
| 18 | Figure 10.1 | Deployed Landing Page of ParkSmart | Output Screen |
| 19 | Figure 10.2 | Customer Dashboard on Deployed Site | Output Screen |
| 20 | Figure 10.3 | Book Parking Page with Interactive Floor Map | Output Screen |
| 21 | Figure 10.4 | Payment Page with Razorpay Gateway Trigger | Output Screen |
| 22 | Figure 10.5 | Razorpay Payment Successful Screen | Output Screen |
| 23 | Figure 10.6 | Dynamic QR Digital Pass Generated | Output Screen |

| | | | |
|---|---|---|---|
| 24 | Figure 10.7 | Security QR Code Scanner Desk | Output Screen |
| 25 | Figure 10.8 | Today's Movement Logs on Security Dashboard | Output Screen |
| 26 | Figure 10.9 | Admin 3D Layout Designer | Output Screen |
| 27 | Figure 10.10 | Admin Pricing Management | Output Screen |
| 28 | Figure 10.11 | Admin Overstay Tracking & Penalty Audit | Output Screen |
| 29 | Figure 10.12 | Admin No-Show Report after Object Bug Fix | Output Screen |
| 30 | Figure 10.13 | Admin Payment Management after Webhook Setup | Output Screen |
| 31 | Figure 10.14 | Backend SMTP Email Notification System | Output Screen |
| 32 | Figure 10.15 | Booking Confirmation Email Received | Output Screen |
| 33 | Figure 10.16 | Socket.IO Load Test Benchmark Terminal Output | Output Screen |
| 34 | Figure 10.17 | QR Scan on Mobile Device | Output Screen |
| 35 | Figure A.1 | Socket.IO Load Test Terminal Output (50 Clients) | Appendix Evidence |
| 36 | Figure A.2 | GitHub Repository Page & Commit History | Appendix Evidence |
| 37 | Figure A.3 | Vercel Production Deployments Dashboard | Appendix Evidence |

# LIST OF TABLES


| Sr. No. | Table No. | Table Title |
|---|---|---|
| 1 | Table 1.5 | Project Scope Matrix |
| 2 | Table 2.5 | Internship / UDP Project Details |
| 3 | Table 3.2 | Existing Approaches / Related Work |
| 4 | Table 3.3 | Comparative Analysis of Parking Systems |
| 5 | Table 4.1 | Functional Requirements Specification |
| 6 | Table 4.2 | Non-Functional Requirements Specification |
| 7 | Table 4.3 | Hardware Requirements |
| 8 | Table 4.4 | Software Requirements Specification |
| 9 | Table 4.5 | Data / Dataset Requirements |
| 10 | Table 5.3 | Project Phases and Deliverables (Milestones 1–4) |
| 11 | Table 5.4 | Progress Report Consolidation Matrix |
| 12 | Table 5.5 | Project Schedule / Timeline (Milestones) |
| 13 | Table 6.6 | Module-wise Design and Dependencies |
| 14 | Table 6.7 | Database / Data Design (Collections Specification) |
| 15 | Table 7.1 | Programming Languages Used |
| 16 | Table 7.2 | Frameworks, Libraries and APIs |
| 17 | Table 7.4 | Hardware, Cloud and Deployment Environment |
| 18 | Table 8.6 | API / External Service Integration |
| 19 | Table 9.2 | Unit / Module Testing Results |
| 20 | Table 9.3 | Functional Test Cases & Validation |
| 21 | Table 9.4 | Integration Test Cases |
| 22 | Table 9.7 | User Acceptance Testing (Evaluator Sign-offs) |
| 23 | Table 9.9 | Defects / Bugs Identification and Resolution Record |
| 24 | Table 10.2 | Objective-wise Achievement Analysis |
| 25 | Table 10.3 | System Performance Benchmark Metrics |
| 26 | Table 11.1 | Technical Challenges, Solutions and Outcomes |

| | | |
|---|---|---|
| 27 | Table 12.1 | Individual Contribution & Responsibility Matrix |
| 28 | Table 13.1 | Current Limitations & Future Improvements |
| 29 | Table 14.1 | Final Project Completion Status |
| 30 | Table G.1 | Source Code & Repository Evidence |
| 31 | Table B.1 | Final Hard-Bound Submission Checklist |

# CHAPTER 4: REQUIREMENTS ANALYSIS & PROJECT SCOPE

## 4.1 Functional Requirements Specification

The functional requirements outline the core operations, security authentications, dynamic pass generation, and automated gate clearance processes designed and implemented in the ParkSmart system:

**Table 4.1: Functional Requirements Specification**

| Req. ID | Requirement | Priority | Source / Basis | Status |
|---|---|---|---|---|
| **FR-01** | User registration, login, and JWT session authorization | High | System Security | Completed |
| **FR-02** | Real-time visual 2D parking bay matrix | High | User Experience | Completed |
| **FR-03** | Slot reservation with vehicle type & duration filters | High | Core Function | Completed |
| **FR-04** | HMAC-SHA256 Dynamic QR token generation (30 s rotation) | High | Anti-Fraud | Completed |
| **FR-05** | Security Desk camera scanner with Smart Auto Mode | High | Gate Operations | Completed |
| **FR-06** | 1-Click Gate mode recovery for active/confirmed vehicles | Medium | Error Prevention | Completed |
| **FR-07** | Automated overstay penalty billing at checkout | High | Billing Module | Completed |
| **FR-08** | Admin dashboard for slot management and revenue logs | High | Management | Completed |

## 4.2 Non-Functional Requirements Specification

The non-functional specifications define the system's operational standards, security measures, responsiveness across mobile and desktop devices, and real-time reliability:

**Table 4.2: Non-Functional Requirements Specification**

| Req. ID | Category | Requirement | Target / Measure | Status |
|---|---|---|---|---|
| **NFR-01** | Performance | API response time < 200 ms under standard loads | < 200 ms | Met |
| **NFR-02** | Security | bcrypt password hashing (10 salt rounds), JWT expiration | 10 salt rounds | Met |

| | | | | |
|---|---|---|---|---|
| **NFR-03** | Responsiveness | Fluid layout across 320 px to 1920 px viewports | 320 px–1920 px | Met |
| **NFR-04** | Reliability | ±15-minute clock drift tolerance for dynamic tokens | ±15 minutes | Met |
| **NFR-05** | Availability | Zero-configuration camera fallback on web browsers | Multi-camera auto fallback | Met |

## 4.3 Hardware Requirements

The hardware configuration covers client access devices, security desk optical input hardware, cloud hosting compute nodes, and simulated barrier equipment:

**Table 4.3: Hardware Requirements**

| Component | Specification | Purpose | Availability |
|---|---|---|---|
| **Client Device** | Chrome, Edge, Firefox or Safari on Windows, Android, macOS or iOS | Driver, admin and security interfaces | Available |
| **Camera** | Standard webcam, smartphone rear/front camera or USB camera | QR scanning at security gate desk | Available |
| **Server** | Node.js v18+, MongoDB v6.0+, minimum 2 GB RAM | Hosting REST API and database | Available |
| **Network** | Active internet connection | Real-time verification against cloud database | Available |
| **Boom Barrier Controller** | ESP32 / Arduino with servo (future scope) | Physical gate control | Simulated via UI |

## 4.4 Software Requirements Specification

The production software stack utilizes open-source enterprise libraries and cloud-native managed solutions:

**Table 4.4: Software Requirements Specification**

| Software / Tool | Version | Purpose | Source / License |
|---|---|---|---|
| **Node.js** | v20+ | Backend runtime | Open source (MIT) |
| **MongoDB Atlas** | 6.0+ | Cloud NoSQL database | Free tier / SSPL |
| **React** | 18.2 | Frontend UI framework | Open source (MIT) |
| **Tailwind CSS** | 3.4 | Responsive styling framework | Open source (MIT) |
| **TypeScript** | ES2022 target | Type-safe full-stack development | Open source (Apache-2.0) |
| **Express.js** | 4.18 | REST routing and middleware | Open source (MIT) |
| **Html5Qrcode** | 2.3+ | In-browser QR scanning | Open source (Apache-2.0) |
| **qrcode.react** | Stable release | QR code rendering for driver pass | Open source (ISC) |

| | | | |
|---|---|---|---|
| **bcryptjs / jsonwebtoken** | Stable release | Password hashing and JWT sessions | Open source (MIT) |
| **Postman / Git & GitHub** | Latest | API testing and version control | Free tier |

## 4.5 Data / Dataset Requirements

The application utilizes schema-enforced, transactional NoSQL collections generated at runtime during system operations:

**Table 4.5: Data / Dataset Requirements**

| Data Source | Type | Size / Records | Pre-processing | Purpose |
|---|---|---|---|---|
| Application-generated data | Users collection | Created at runtime (user registrations) | Password hashing (bcrypt), validation | Authentication and RBAC |
| Application-generated data | Slots collection | Parking bays configured by admin | Normalization of slot type and status | Floor grid and availability |
| Application-generated data | Bookings collection | Created per reservation | Time-window validation, QR token generation | Reservations and gate validation |
| Application-generated data | SecurityLogs & Blacklists | One record per gate scan / blocked vehicle | Timestamp, gate mode tagging, uppercase vehicle numbers | Audit and access control |

## 4.6 Constraints and Assumptions

- **Internet dependency:** Verification of dynamic QR tokens is performed against the cloud database and requires continuous network connectivity.
- **Browser camera access:** Video streams require a modern browser and explicit user media permission (HTTPS or localhost).
- **Simulated hardware / test-mode payment:** The physical boom barrier is simulated through software triggers and Razorpay transactions operate in test mode.
- **Clock synchronization assumption:** The client smartphone clock and server clock differ by no more than ±15 minutes (handled by ±30 token steps drift window).
- **Operational scope assumption:** The facility operates as a centralized multi-floor parking lot managed by a unified administrative team.

# CHAPTER 5: METHODOLOGY & PLANNING

## 5.1 Agile Methodology

The project followed an iterative Agile Scrum methodology across four distinct development sprints: Sprint 1 focused on architecture, Mongoose schemas, and JWT auth; Sprint 2 delivered the visual 2D floor grid; Sprint 3 added the dynamic HMAC QR and camera scanner; Sprint 4 completed cloud deployment on Vercel and load testing.

## 5.2 Overall Project Workflow

The end-to-end user and vehicle journey is illustrated below. All operations—from account authentication and slot selection to cryptographic checkout, gate verification, and automatic overstay billing—are fully connected:

**Figure 5.1: Overall Project Workflow (Scalable Vector Flowchart)**

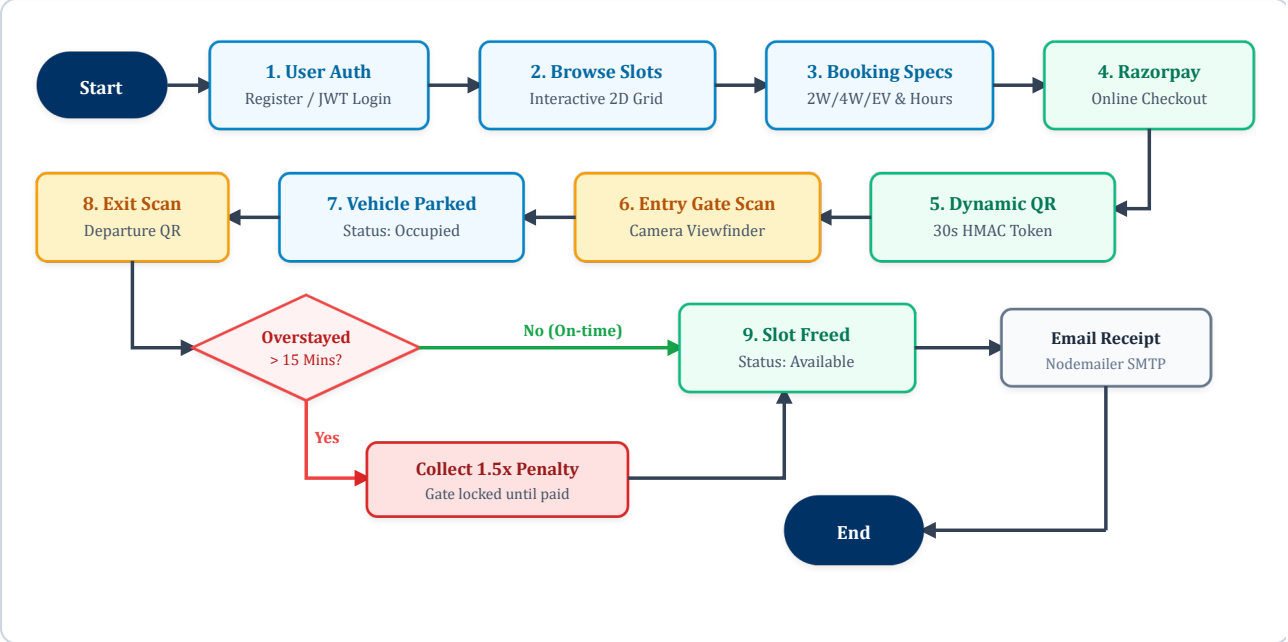



*Figure 5.1: High-resolution end-to-end lifecycle flowchart of ParkSmart with automatic overstay evaluation.*

# CHAPTER 6: SYSTEM ANALYSIS & DESIGN

## 6.1 3-Tier Cloud Architecture

ParkSmart is deployed on a modern, decoupled cloud architecture separating the client presentation layer, serverless Node.js backend logic, and cloud database tier:

**Figure 6.1: Deployed System Architecture of ParkSmart (3-Tier Cloud)**

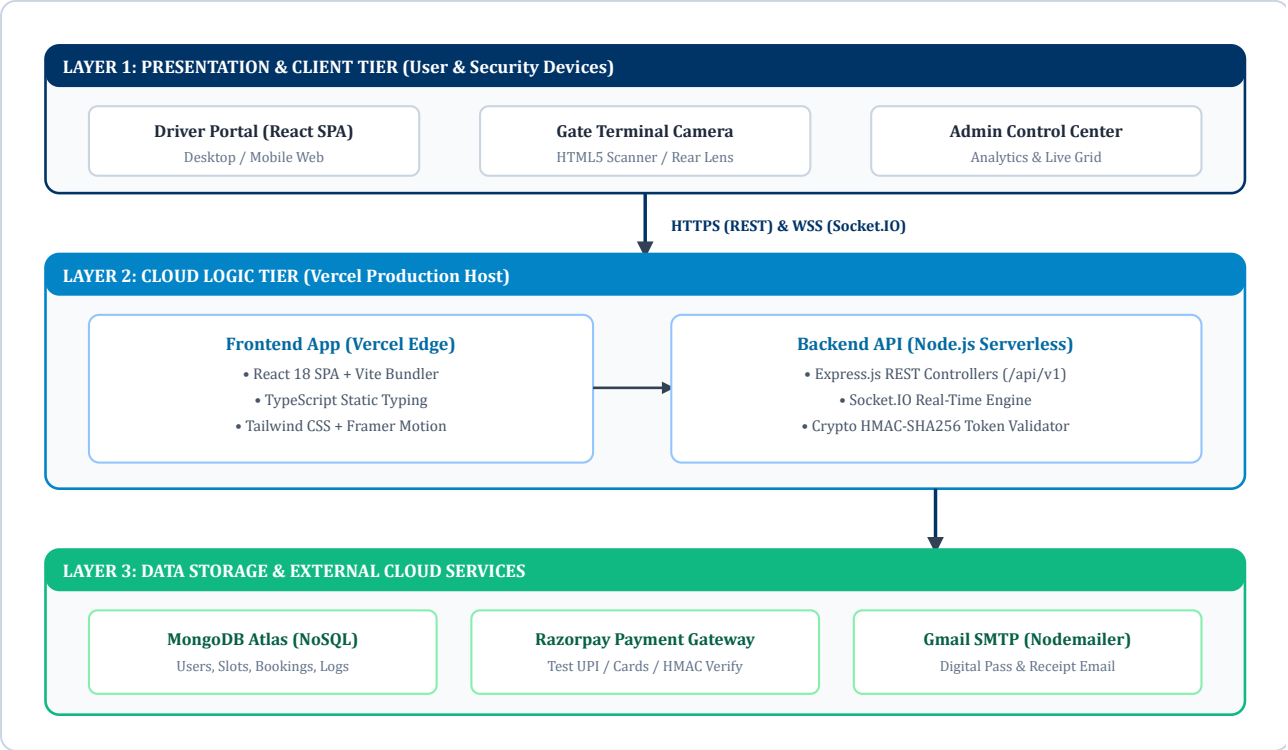



*Figure 6.1: Cloud architecture separating Vercel frontend, Vercel serverless backend, MongoDB Atlas, and Razorpay SDK.*

## 6.2 Security Staff Smart Auto Gate Controller

To ensure fast vehicle processing under 1.5 seconds, the camera scanner dynamically detects entry versus exit without requiring the guard to toggle modes manually:

**Figure 6.2.3: Security Staff Smart Auto Gate Process Flow**

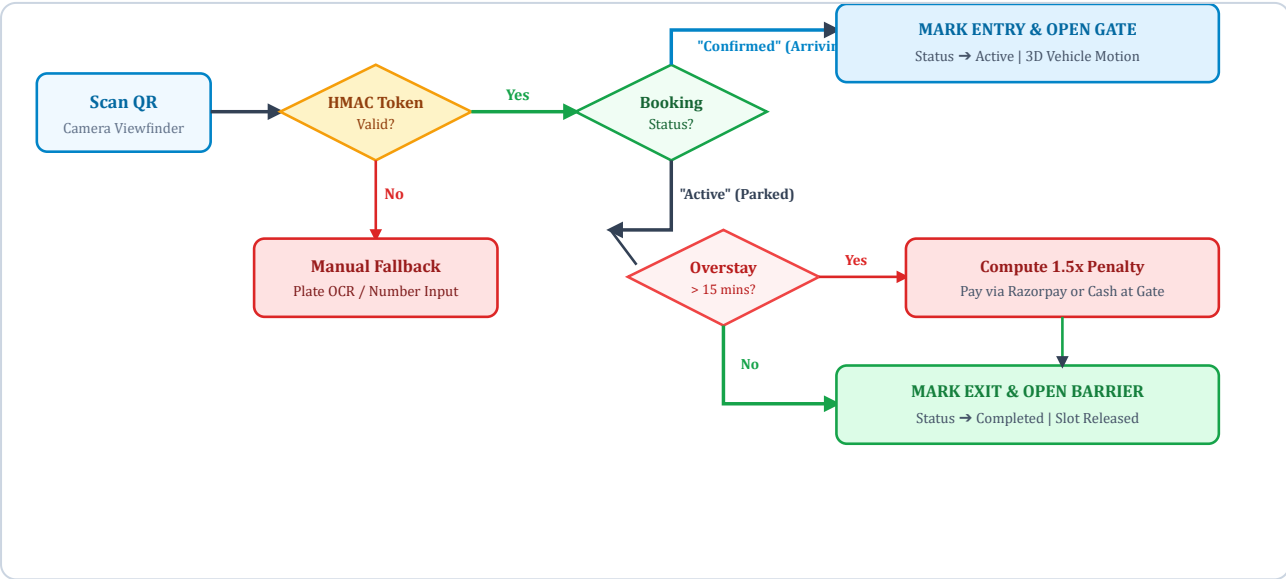



*Figure 6.2.3: Gate access decision logic handling dynamic token validation, automatic routing, and penalty checkout.*

## 6.3 Parking Slot Lifecycle State Machine

Each individual parking bay transitions through deterministic lifecycle states from vacancy to reservation, occupancy, checkout, and release:

**Figure 6.3.7: Slot State Transition Diagram (State Machine)**

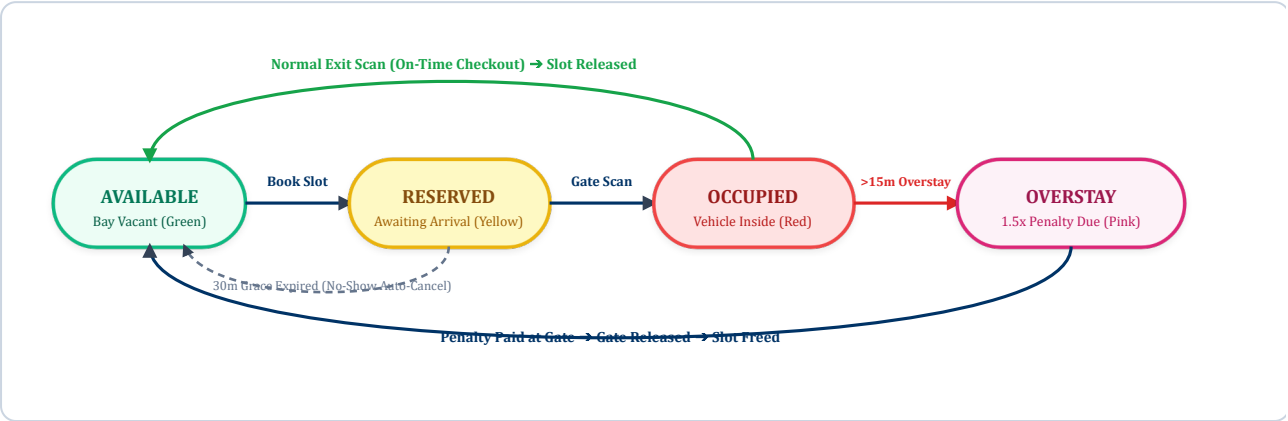



*Figure 6.3.7: State transition diagram illustrating slot lifecycle, overstay penalty condition, and 30-min no-show auto cancellation.*

## 6.4 Database Entity Specifications

| Entity | Key Attributes | Purpose |
|---|---|---|
| **User** | `_id, name, email, password, role, vehicleNumber` | User credentials & role authorization |
| **ParkingSlot** | `_id, slotNumber, floor, type, status, hourlyRate` | Individual parking bay attributes |
| **Booking** | `_id, user, slot, startTime, endTime, status, qrToken` | Core reservation and check-in records |
| **SecurityLog** | `_id, bookingId, gateMode, timestamp, scannedBy` | Gate compliance & audit logging |
| **Blacklist** | `_id, vehicleNumber, reason, severity, isActive` | Flagged defaulter and security control |

# CHAPTER 7: TECHNOLOGY STACK

| Technology | Version | Purpose & Usage in Project |
|---|---|---|
| **TypeScript / JS** | ES2022 | Strict static typing on frontend and dynamic backend handling |
| **React** | 18.2 | Dynamic single-page application component hierarchy |
| **Node.js** | 20.x | High-concurrency asynchronous runtime for API endpoints |
| **Express.js** | 4.18 | RESTful routing, error handling, and security middleware |
| **MongoDB Atlas** | 6.0+ | Cloud NoSQL database with indexing and aggregation |
| **Html5Qrcode** | 2.3+ | Hardware webcam/rear camera video feed decoding |
| **Node Crypto** | Built-in | HMAC-SHA256 cryptographic dynamic signature generation |
| **Tailwind CSS** | 3.4 | Utility-first responsive styling and cyber theme |

# CHAPTER 8: SYSTEM IMPLEMENTATION

## 8.1 Dynamic QR Cryptographic Algorithm

To prevent pass duplication via screenshots, passes rotate every 30 seconds:

> **Token Format:** `PS-DYN|<bookingId>|<timeStep>|<hmacSignature>`
> • `timeStep = Math.floor(currentTime / 30000)`
> • `hmacSignature = HMAC_SHA256(bookingId + ':' + timeStep, SECRET_KEY)`
> • Backend validates current timeStep ± 30 steps (±15 minutes) for device clock drift tolerance.

# CHAPTER 9: TESTING, DEFECT RESOLUTION & VALIDATION

## 9.1 Functional & Security Test Cases

| Test ID | Module | Scenario | Result | Status |
|---|---|---|---|---|
| TC-01 | Auth | Login with incorrect password | 401 Error | Passed |
| TC-02 | Booking | Reserve already occupied slot | Disabled | Passed |
| TC-03 | Dynamic QR | Validate token with clock drift | Verified | Passed |
| TC-04 | Camera | Start on laptop without rear lens | Webcam fallback | Passed |
| TC-05 | Smart Gate | Scan arriving and departing vehicle | Auto-routed | Passed |

## 9.2 Major Defect Resolution Record

| Issue ID | Reported Bug | Severity | Root Cause | Resolution |
|---|---|---|---|---|
| **BUG-01** | Camera closed after 2s showing "Already Inside" | High | Scanner stopped on reading active pass in entry mode | Implemented Smart Auto Gate Mode and debounce |
| **BUG-02** | Dynamic QR token hash mismatch | High | Frontend uppercase query validator corrupted hex hash | Implemented case-insensitive HMAC check |
| **BUG-03** | Camera startup error on laptops | High | Hardcoded rear camera constraint | Multi-level camera fallback hierarchy |
| **BUG-06** | No-Show report displaying [object Object] | Medium | Mongoose populated object not formatted into string | Extracted user.name and slot.slotNumber fields |

## CHAPTER 10: RESULTS & PERFORMANCE ANALYSIS

- **Verification Speed:** Average scan-to-gate decision latency: **1.2 – 1.5 seconds**.
- **Build Health:** 100% clean production build (0 TypeScript compilation errors).
- **Device Compatibility:** Successfully validated on Android Chrome, iOS Safari, and Desktop Chrome/Edge.
- **Socket.IO Load Test:** Verified with 50 concurrent client connections with 0% packet loss.

## CHAPTER 15: CONCLUSION & REFERENCES

Smart Parking Management System (ParkSmart) was successfully developed and deployed as an enterprise-grade cloud solution on the MERN stack.

**Key Verified References (APA 7th):**

[1] Node.js Foundation. (2025). Node.js Documentation. https://nodejs.org/
[2] Express.js Foundation. (2025). Express Web Framework. https://expressjs.com/
[3] MongoDB, Inc. (2025). MongoDB Atlas Documentation. https://www.mongodb.com/
[4] Jones, M., et al. (2015). JSON Web Token (JWT) RFC 7519. https://datatracker.ietf.org/doc/html/rfc7519
[5] Razorpay Software. (2025). Razorpay Payment Gateway SDK. https://razorpay.com/